

SQLdataclass

SQLDataclass

Memory-efficient ORM bridge: pydantic dataclasses + SQLAlchemy Core. FastAPI compatible.

Define your models once — like SQLAlchemy — but get the memory footprint of plain dataclasses. SQLDataclass uses pydantic dataclasses (`slots=True`) under the hood, with full pydantic validation, relationships, and native FastAPI support.

Performance

All benchmarks run on SQLite with 10,000 rows and 20 fields per row.
Reproducible via `src/sqldataclass/tests/performance_tests/`.

Object construction (20 fields, 10k objects, no DB)

Approach	B/row	Time
dict	578	22 ms
stdlib dataclass (<code>slots=True</code>)	306	27 ms
pydantic dataclass (<code>slots=True</code>)	306	191 ms
SQLDataclass	322	66 ms
SQLAlchemy ORM	1,690	246 ms
Pydantic BaseModel	2,914	64 ms
SQLDataclass <code>SQLModel</code>	2,914	67 ms
SQLModel (tiangolo)	4,538	916 ms

Database loading — SQLite (10k rows, 20 fields)

Approach	B/row	Time
Raw SQL → dict	963	124 ms
Raw SQL → stdlib dataclass	691	136 ms

Approach	B/row	Time
Raw SQL → pydantic dataclass	691	307 ms
SQLDataclass load_all	708	211 ms
SQLDataclass SQLAlchemy load_all	1,204	170 ms
SQLAlchemy ORM Session.query	2,098	167 ms
SQLModel (tiangolo) session.exec	2,410	169 ms

Times include query execution + object construction. Both `SQLDataclass` and `SQLModel load_all` bypass pydantic's `__init__` overhead — `SQLDataclass` uses `validate_python` (Rust fast path), `SQLModel` uses direct `__dict__` hydration (like the SA ORM does). `SQLModel load_all` uses **1.7x less memory than SA ORM** and **2x less than tiangolo's SQLModel** while matching their speed.

Complex models with relationships (100 teams, 5k heroes, 20 tags, SQLite)

Teams with heroes (one-to-many, 100 teams + 5k heroes):

Library	Memory	Load time	Notes
SQLDataclass	1.2 MB	13 ms	Two-query + back-ref stitching, no session
SQLAlchemy ORM + joinedload	8.1 MB	19 ms	JOIN-based, needs session

Heroes with team + tags (many-to-one + many-to-many, 5k heroes, 20 tags, 3 tags/hero):

Library	Memory	Load time	Notes
SQLDataclass	3.7 MB	53 ms	PK-cached deduplication, no session
SQLAlchemy ORM + eager	12.3 MB	70 ms	Identity map deduplication

When to use what

- **Simple/flat models** — `SQLDataclass` wins on both memory and speed (3-14x less memory)
- **One-to-many** — `SQLDataclass` wins (6.7x less memory, faster, with automatic back-references)
- **Many-to-many** — `SQLDataclass` wins (3.3x less memory, faster, PK-based deduplication)

Summary

Benchmark	SQLDataclass	SQLDataclass SQLModel	vs SQLAlchemy ORM	vs SQLModel (tiangolo)
DB loading (memory)	3x less	1.7x less	baseline	1.1x less
DB loading (speed)	same	same	baseline	same
One-to-many (memory)	6.7x less	—	baseline	—
Many-to-many (memory)	3.3x less	—	baseline	—
Object construction (memory)	5x less	1.7x more	baseline	1.6x less
Object construction (speed)	4x faster	4x faster	baseline	14x faster

SQLDataclass vs SQLDataclass SQLModel : For DB loading, `SQLModel` uses direct `__dict__` hydration (like the SA ORM) — **1,204 B/row**, beating both SA ORM (2,098) and tiangolo (2,410). For object construction (no DB), `SQLModel` uses 2,914 B/row due to `BaseModel.__dict__` overhead vs `SQLDataclass`'s 322 B/row (`slots=True`). Use `SQLDataclass` for maximum memory efficiency; use `SQLModel` when you need the full `BaseModel` API (`model_dump` , `model_validate` , JSON schema, etc.).

Why the difference?

- **`slots=True` pydantic dataclasses** — no `__dict__` , minimal per-instance overhead
- **`validate_python` fast path** — bypasses pydantic's `__init__` wrapper, 40% faster than `cls(**row)`
- **SQLAlchemy Core, not ORM** — no session, no identity map, no state tracking overhead
- **PK-cached M2M deduplication** — same tag = same instance, matching ORM's identity map benefit
- **Two-query collections** — one-to-many loaded via `WHERE fk IN (...)` , not expensive JOINS
- **Back-reference stitching** — `hero.team = parent` set directly, zero extra queries

Install

```
pip install sqldataclass
```

For PostgreSQL upsert support:

```
pip install sqldataclass[postgres]
```

Quick start

Define your model

One class. That's it. No separate schema and domain model.

```
from sqldataclass import SQLDataclass, Field

class Hero(SQLDataclass, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None
```

This creates:

- A **pydantic dataclass** with `slots=True` (validation, FastAPI compat, minimal memory)
- A **SQLAlchemy Table** (for DDL and queries, never instantiated)

Setup

```
from sqlalchemy import create_engine

engine = create_engine("sqlite:///app.db")
SQLDataclass.metadata.create_all(engine)

# Bind the engine once – conn becomes optional everywhere
SQLDataclass.bind(engine)
```

Insert data

```
hero = Hero(name="Spider-Man", secret_name="Peter Parker")
hero.insert()

# Bulk insert
```

```

Hero.insert_many(objects=[
    Hero(name="Iron Man", secret_name="Tony Stark", age=45),
    Hero(name="Thor", secret_name="Thor Odinson", age=1500),
])

```

Query data

```

# Load all
heroes = Hero.load_all()

# Filter
heroes = Hero.load_all(where=Hero.c.age > 100)

# Load one
hero = Hero.load_one(where=Hero.c.name == "Spider-Man")

# Order
heroes = Hero.load_all(where=Hero.c.age > 25, order_by=Hero.c.name)

```

Explicit connections (when you need transaction control)

```

with engine.begin() as conn:
    hero1.insert(conn)
    hero2.insert(conn)
    # both commit together, or both rollback on error

```

Working with existing databases

If your tables already exist (managed by Alembic, a DBA, or another service), skip `metadata.create_all()` — just define models that match the existing schema:

```

from sqlalchemy import SQLAlchemy, Field

# Match your existing table and column names exactly
class User(SQLAlchemy, table=True):
    __tablename__ = "users" # must match the existing table name
    id: int = Field(primary_key=True)
    email: str
    name: str
    is_active: bool = True

engine = create_engine("postgresql+psycopg2://user:pass@host/mydb")
SQLAlchemy.bind(engine)

# No create_all() needed — read and write directly

```

```
users = User.load_all(where=User.c.is_active == True)
User.update({"is_active": False}, where=User.c.id == 42)
```

You don't need to define every column — only the ones you read or write. Columns not in your model are simply ignored.

Migrations with Alembic

SQLDataclass uses standard SQLAlchemy `MetaData`, so [Alembic](#) works out of the box for schema migrations.

Setup:

```
pip install alembic
alembic init migrations
```

Edit `migrations/env.py` — point Alembic at your SQLDataclass metadata:

```
from sqldataclass import SQLDataclass

# Import your models so they register their tables
from myapp.models import Hero, Team # noqa: F401

target_metadata = SQLDataclass.metadata
```

Generate and run migrations:

```
# Auto-generate migration from model changes
alembic revision --autogenerate -m "add heroes table"

# Apply migration
alembic upgrade head
```

Example: adding a column to an existing model

Start with:

```
class Hero(SQLDataclass, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
```

Later, add a field:

```
class Hero(SQLDataclass, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    # ... other fields
```

```
power: str = ""           # new column
age: int | None = None    # new nullable column
```

Then generate and apply the migration:

```
alembic revision --autogenerate -m "add power and age to hero"
alembic upgrade head
```

Alembic auto-detects the diff and generates:

```
def upgrade():
    op.add_column('hero', sa.Column('power', sa.String(), nullable=False))
    op.add_column('hero', sa.Column('age', sa.Integer(), nullable=True))

def downgrade():
    op.drop_column('hero', 'age')
    op.drop_column('hero', 'power')
```

This works for any schema change: adding/removing columns, renaming tables, changing types, adding indexes, etc.

Use `create_all()` for development/testing and Alembic for production deployments.

Relationships

SQLDataclass supports all common relationship patterns, loaded eagerly via SQLAlchemy Core — no ORM session required.

Many-to-one

```
from sqldataclass import SQLDataclass, Field, Relationship

class Team(SQLDataclass, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str

class Hero(SQLDataclass, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    team_id: int = Field(foreign_key="team.id")
    team: Team | None = Relationship() # auto-JOINed on load

hero = Hero.load_one(where=Hero.c.name == "Spider-Man")
print(hero.team.name) # "Avengers"
```

One-to-many

```
class Team(SQLDataclass, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    heroes: list[Hero] = Relationship(back_populates="team")

team = Team.load_one(where=Team.c.name == "Avengers")
print([h.name for h in team.heroes]) # ["Iron Man", "Thor"]
```

One-to-many uses a **two-query strategy** (not JOIN-then-group) for memory efficiency — one query for parents, one `WHERE fk IN (...)` for all children.

Many-to-many

```
class HeroTeamLink(SQLDataclass, table=True):
    hero_id: int = Field(primary_key=True, foreign_key="hero.id")
    team_id: int = Field(primary_key=True, foreign_key="team.id")

class Hero(SQLDataclass, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    teams: list[Team] = Relationship(link_model=HeroTeamLink)

class Team(SQLDataclass, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    heroes: list[Hero] = Relationship(link_model=HeroTeamLink)

hero = Hero.load_one(where=Hero.c.name == "Wolverine")
print([t.name for t in hero.teams]) # ["Avengers", "X-Men"]
```

Discriminated unions

For polymorphic data stored in separate tables:

```
from typing import Literal

class NormalData(SQLDataclass, table=True):
    id: int = Field(primary_key=True, foreign_key="participant.id")
    behavior: Literal["normal"] = "normal"
    p_max: float = 0.0

class BatteryData(SQLDataclass, table=True):
    id: int = Field(primary_key=True, foreign_key="participant.id")
    behavior: Literal["battery"] = "battery"
    capacity: float = 0.0

class Participant(SQLDataclass, table=True):
```



```

    id: int | None = Field(default=None, primary_key=True)
    name: str
    behavior: str # discriminator column
    data: NormalData | BatteryData = Relationship(discriminator="behavior")

p = Participant.load_one(where=Participant.c.name == "Alice")
print(type(p.data).__name__) # "NormalData"
print(p.data.p_max)          # 100.0

```

Single-table inheritance

Store multiple subtypes in one table with a discriminator column. Child classes can add their own fields (auto-appended as nullable columns):

```

class Vehicle(SQLDataclass, table=True):
    __discriminator__ = "type" # enables single-table inheritance
    id: int | None = Field(default=None, primary_key=True)
    type: str = ""
    name: str = ""

class Car(Vehicle):
    # just inherit – no extra keywords
    # auto-added to Vehicle's table
    doors: int | None = None

class Truck(Vehicle):
    # auto-added to Vehicle's table
    payload: float | None = None

```

```

# Insert – discriminator auto-set from class name
Car(name="Civic", doors=4).insert() # type="car"
Truck(name="F-150", payload=1000).insert() # type="truck"

# Subtype queries – auto-filtered
cars = Car.load_all() # only cars
trucks = Truck.load_all() # only trucks

# Polymorphic query – returns correct subtypes
all_vehicles = Vehicle.load_all() # [Car(...), Truck(...), ...]
type(all_vehicles[0]) # <class 'Car'>

# Scoped update/delete
Car.update({"doors": 2}, where=Car.c.name == "Civic")
Truck.delete() # only deletes trucks

```

Override the default discriminator value with `__discriminator_value__` :

```

class Motorcycle(Vehicle):
    __discriminator_value__ = "moto" # instead of default "motorcycle"
    wheel_count: int | None = None

```

Use with FastAPI

Pydantic dataclasses are first-class citizens in FastAPI — no conversion needed:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/heroes", response_model=list[Hero])
def get_heroes():
    return Hero.load_all()

@app.get("/heroes/{hero_id}", response_model=Hero)
def get_hero(hero_id: int):
    return Hero.load_one(where=Hero.c.id == hero_id)
```

Data-only models (API schemas)

Models without `table=True` are pure pydantic dataclasses — useful for request bodies:

```
class HeroCreate(SQLDataclass):
    name: str
    secret_name: str
    age: int | None = None

@app.post("/heroes", response_model=Hero)
def create_hero(data: HeroCreate):
    hero = Hero(name=data.name, secret_name=data.secret_name, age=data.age)
    hero.insert()
    return hero
```

Field options

`Field()` accepts both pydantic and SQLAlchemy parameters:

```
class User(SQLDataclass, table=True):
    id: int | None = Field(default=None, primary_key=True)
    email: str = Field(unique=True, index=True, max_length=255)
    name: str = Field(min_length=1, max_length=100)
    age: int = Field(ge=0, le=200)
    team_id: int | None = Field(default=None, foreign_key="teams.id")
    # Not stored in DB – only exists on the Python object
    display_name: str = Field(default="", column=False)
    is_cached: bool = Field(default=False, column=False)
```

Parameter	Type	Description
<code>primary_key</code>	<code>bool</code>	Mark as primary key
<code>index</code>	<code>bool</code>	Create database index
<code>unique</code>	<code>bool</code>	Add unique constraint
<code>foreign_key</code>	<code>str</code>	Foreign key reference (e.g. <code>"users.id"</code>)
<code>nullable</code>	<code>bool</code>	Override nullable inference
<code>sa_type</code>	<code>TypeEngine</code>	Override SQLAlchemy column type
<code>default</code>	<code>Any</code>	Default value
<code>ge</code> , <code>le</code> , <code>gt</code> , <code>lt</code>	<code>float</code>	Pydantic numeric validators
<code>min_length</code> , <code>max_length</code>	<code>int</code>	Pydantic string validators
<code>pattern</code>	<code>str</code>	Pydantic regex pattern
<code>column</code>	<code>bool</code>	<code>False</code> = field exists on Python object but not in DB

Relationship options

`Relationship()` marks a field as loaded from a related table — not stored as a column.

Parameter	Type	Description
<code>back_populates</code>	<code>str</code>	Inverse field name on the child model
<code>link_model</code>	<code>type</code>	Link table class for many-to-many
<code>discriminator</code>	<code>str</code>	Column name for discriminated unions
<code>order_by</code>	<code>str</code>	Column name to sort collection children by
<code>default</code>	<code>Any</code>	Default value (<code>None</code> for many-to-one, <code>[]</code> for collections)

Custom type annotations

`SQLDataclass` doesn't bundle domain-specific type annotations (e.g. `numpy`), but you can define your own in your project and use them seamlessly with pydantic's `Annotated` types:

```

# your_project/annotations.py
from functools import partial
from typing import Annotated

import numpy as np
import numpy.typing as npt
from pydantic import BeforeValidator, PlainSerializer

def _to_np_array(dtype, x):
    return np.asarray(x, dtype=dtype)

class Np:
    """Numpy type annotations with auto-serialization."""

    float64 = Annotated[
        np.float64,
        PlainSerializer(float, return_type=float, when_used="always"),
        BeforeValidator(np.float64),
    ]
    int64 = Annotated[
        np.int64,
        PlainSerializer(int, return_type=int, when_used="always"),
        BeforeValidator(np.int64),
    ]

    class Array:
        float64 = Annotated[
            npt.NDArray[np.float64],
            PlainSerializer(lambda x: x.tolist(), return_type=list[float],
                when_used="always"),
            BeforeValidator(partial(_to_np_array, np.float64)),
        ]

```

Then use them in your models:

```

from sqlalchemy import SQLDataclass
from your_project.annotations import Np

class Measurement(SQLDataclass):
    score: Np.float64
    readings: Np.Array.float64

m = Measurement(score=9.5, readings=[1.0, 2.0, 3.0])
m.dump() # {"score": 9.5, "readings": [1.0, 2.0, 3.0]}

```

This pattern works for any custom type — numpy, pandas, domain objects, etc. Pydantic's `Annotated` + `BeforeValidator` / `PlainSerializer` handles the conversion automatically.

API reference

Model methods

All methods accept an optional `conn` parameter. If omitted, a connection is auto-created from the bound engine (see `SQLDataclass.bind(engine)`).

Method	Type	Description
<code>SQLDataclass.bind(engine)</code>	classmethod	Bind engine — makes <code>conn</code> optional everywhere
<code>Model.select()</code>	classmethod	Build a <code>SELECT</code> query
<code>Model.load_all(conn=, where=, order_by=)</code>	classmethod	Load all matching rows with relationships
<code>Model.load_one(conn=, where=)</code>	classmethod	Load one row or <code>None</code>
<code>Model.insert_many(conn=, objects=)</code>	classmethod	Bulk insert
<code>Model.update(values, conn=, where=)</code>	classmethod	Update matching rows, returns count
<code>Model.delete(conn=, where=)</code>	classmethod	Delete matching rows, returns count
<code>instance.insert(conn=)</code>	instance	Insert this row
<code>instance.to_dict(exclude_keys=)</code>	instance	Flat dict for SQL
<code>instance.upsert(conn=, index_elements=)</code>	instance	PostgreSQL upsert
<code>Model.c</code>	attribute	Column access for <code>WHERE</code> clauses
<code>Model.metadata</code>	attribute	SQLAlchemy MetaData

Low-level bridge API

For advanced use cases, the underlying bridge functions are also available:

Function	Description
<code>load_all(conn, query, cls)</code>	Execute query, construct instances inline
<code>fetch_all(conn, query)</code>	Execute query, return <code>list[dict]</code>
<code>fetch_one(conn, query)</code>	Execute query, return single <code>dict</code> or <code>None</code>

Function	Description
<code>insert_row(conn, table_class, values)</code>	Insert a single row
<code>insert_many(conn, table_class, rows)</code>	Insert multiple rows
<code>flatten_for_table(obj, exclude_keys=)</code>	Flatten dataclass to dict
<code>nest_fields(data, field_name, keys)</code>	Reshape flat dict for nested models
<code>format_discriminated(data, cls, ...)</code>	Reshape flat row for discriminated unions

Design philosophy

1. **One class, one definition** — no separate SQL schema and domain model
2. **Memory-first** — pydantic dataclasses with `slots=True` match stdlib dataclass footprint
3. **SQLAlchemy Core, not ORM** — explicit queries, no hidden state tracking
4. **Relationships without a session** — eager loading via JOINS and two-query strategy
5. **FastAPI native** — pydantic dataclasses work as response models out of the box
6. **Escape hatches** — low-level bridge API available when you need full control

Known limitations

SQLDataclass intentionally trades some ORM features for memory efficiency and simplicity. Here's what it doesn't do (yet):

Limitation	Workaround
No lazy loading (by design) — relationships are always eager-loaded	Use <code>.select()</code> + low-level <code>load_all()</code> to control what's loaded
No <code>update()</code> or <code>delete()</code> methods	Fixed in v0.0.6
No pagination in <code>load_all</code>	Fixed in v0.0.7 — <code>Hero.load_all(limit=10, offset=20)</code>
No nested relationship loading	Fixed in v0.1.0 — <code>hero.team.league</code> auto-loads recursively
No relationship ordering	Fixed in v0.0.8 — <code>Relationship(order_by="name")</code>
No single-table inheritance	Fixed in v0.1.1 — <code>class Car(Vehicle): with __discriminator__</code>
Composite PKs don't work with collection relationships	Fixed in v0.1.0

Limitation	Workaround
No identity map (by design) — loading the same row twice creates separate objects	Immutable dataclass pattern; cache at application level if needed
<code>bind()</code> is global	Fixed in v0.0.9 — <code>Hero.bind(engine_a)</code> , <code>Team.bind(engine_b)</code>
Eager-only collections (by design) — one-to-many/many-to-many always load all children	Filter at query level or use low-level bridge API

Acknowledgements

SQLDataclass was born from combining two lines of work:

SQLModel by Sebastián Ramírez and its contributors provided the inspiration for the single-class API — one model definition that serves as both the database schema and the pydantic data model. SQLDataclass recreates this developer experience while targeting lower memory consumption by building on pydantic dataclasses and SQLAlchemy Core instead of the full ORM.

Prior research by Marcel Spitz, Kai Habermann, and Christian Siebert on pydantic memory and runtime performance — conducted in a separate project with Lennart Jansen — established that pydantic dataclasses with `slots=True` achieve the same memory footprint as stdlib dataclasses while retaining full validation. These findings (12.8x less memory than BaseModel, 5.9x less than ORM instances) became the empirical foundation for SQLDataclass's architecture. The goal was to combine these results with SQLModel's ergonomics.

Requirements

- Python 3.13+
- pydantic >= 2.0
- sqlalchemy >= 2.0

License

MIT